

1 Gymnasium Environment Adaptation

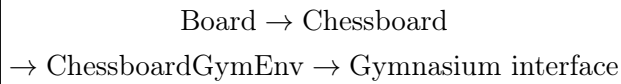
1.1 Motivation

The initial reinforcement learning environment was implemented as a custom `Chessboard` class. The environment was designed specifically for the developed grid-world and therefore did not depend on a particular reinforcement learning library. It manages the board, state transitions, terminal conditions, and reward calculation through the underlying `Board`, `CellBox`, and `Chessboard` classes.

Although this architecture provides direct control over the environment semantics, it does not directly follow the interface expected by Gymnasium. In particular, a Gymnasium environment must provide standard `observation_space` and `action_space` objects and implement the `reset()` and `step()` methods.

Therefore, the existing environment was not replaced. Instead, a Gymnasium adaptation layer was introduced between the custom environment and the Gymnasium API. This preserves the existing environment implementation while allowing it to be used with standard reinforcement learning tooling, vectorised environments, and the developed PPO algorithm.

The resulting architecture is:



The `Chessboard` remains responsible for the environment dynamics, while `ChessboardGymEnv` translates these dynamics into the representation required by Gymnasium.

1.2 Adaptation of the Custom Environment

The Gymnasium implementation was designed as a wrapper around the existing environment rather than as an independent implementation.

The original environment provides the transition

$$(s_t, a_t) \rightarrow (s_{t+1}, r_t, \text{done}),$$

where s_t denotes the internal environment state and a_t the action. The Gymnasium adapter translates this interaction into the standard Gymnasium interface:

$$(s_t, a_t) \rightarrow (o_{t+1}, r_t, \text{terminated}, \text{truncated}, \text{info}).$$

The observation is obtained from the existing state encoding function,

$$o_t = \phi(s_t),$$

where ϕ maps the internal state to the numerical representation provided to the learning algorithm. Thus, the Gymnasium layer does not introduce a new state representation; it exposes the existing representation through the standard observation interface.

This separation allows the internal board and transition logic to remain unchanged when the observation representation is modified.

1.3 Generic Gymnasium Environment Base Class

A generic `GymEnv` base class was implemented to centralise the construction of Gymnasium spaces. Rather than defining the observation and action spaces independently for every environment, the class receives configurations describing the required spaces.

The implementation currently supports:

- `Discrete`,
- `Box`,
- `MultiBinary`,
- `MultiDiscrete`,
- `Dict`.

The purpose of this class is to provide a common Gymnasium interface for future custom environments while leaving environment-specific behaviour to their respective subclasses.

Conceptually,

$$\text{GymEnv} = \text{common Gymnasium interface} + \text{space construction}.$$

1.4 ChessboardGymEnv

The `ChessboardGymEnv` class provides the concrete adaptation of the custom `Chessboard` environment.

It receives:

1. an existing `Chessboard` environment;
2. a state-to-vector function $\phi(s)$.

The observation space is derived from the dimensionality of the state representation:

$$n_{\text{features}} = |\phi(s)|.$$

For the current grid-world implementation, the observation is represented as a one-dimensional floating-point vector and exposed through a Gymnasium `Box` space.

The action space is derived from the number of actions available in the original environment:

$$|\mathcal{A}| = |\text{chessboard.actions}|.$$

The Gymnasium action space is therefore represented as

$$\mathcal{A}_{\text{Gym}} = \{0, 1, \dots, |\mathcal{A}| - 1\}.$$

Each integer is mapped internally to the corresponding action used by the original **Chessboard**. For example,

$$0 \rightarrow \text{up}, \quad 1 \rightarrow \text{right}, \quad 2 \rightarrow \text{down}, \quad 3 \rightarrow \text{left}.$$

The PPO agent therefore interacts only with the Gymnasium representation, while the original action representation remains internal to the **Chessboard**.

1.5 Reset and Step Translation

The adapter implements the two principal Gymnasium operations, **reset()** and **step()**.

During reset, the underlying **Chessboard** is reset first. The resulting state is then converted into an observation:

$$o_0 = \phi(s_0).$$

The adapter also supports an optional random-start mechanism. When enabled, the initial state can be sampled from free states, with states adjacent to traps receiving increased sampling probability. This provides additional exposure to more difficult starting configurations during training.

During a step, the interaction follows

$$\begin{aligned} a_{\text{gym}} &\rightarrow a_{\text{chessboard}} \rightarrow \text{Chessboard.step()} \\ &\rightarrow s_{t+1} \rightarrow \phi(s_{t+1}). \end{aligned}$$

Thus, the adapter performs three main translations:

1. Gymnasium action $\rightarrow$ original environment action;
2. internal state $\rightarrow$ observation vector;
3. internal termination information $\rightarrow$ **terminated**, **truncated**, and **info**.

The **info** dictionary can additionally expose environment-specific information such as the state name, cell symbol, and terminal outcome. This allows evaluation and debugging code to access internal environment information without including it in the observation given to the policy.

1.6 Separation of Environment Dynamics and Reward

An important objective of the Gymnasium implementation is to make the reward mechanism independently replaceable.

The original **Chessboard** already allows a reward function to be provided directly. The Gymnasium implementation extends this mechanism with a dedicated **RewardCustom** wrapper.

The wrapper inherits from Gymnasium’s **Wrapper** class and intercepts the transition

$$(o_t, a_t) \rightarrow (o_{t+1}, r_t, \text{terminated}, \text{truncated}, \text{info}).$$

It then replaces the original reward with the value returned by an externally supplied reward function:

$$r'_t = R_{\text{custom}}(s_t, a_t, s_{t+1}, \dots)$$

The original reward is retained in the `info` dictionary under `original_reward`. This makes it possible to compare the reward generated by the base environment with the reward supplied by the wrapper.

1.7 Flexible Reward Function Interface

The reward wrapper was designed to avoid imposing a single rigid reward function signature. Depending on the task, a reward function may use information such as:

- current state;
- action;
- next state;
- current observation;
- next observation;
- original reward;
- termination flags;
- information dictionary;
- underlying environment.

Conceptually, the reward interface can therefore be represented as

$$R : (s_t, a_t, s_{t+1}, o_t, o_{t+1}, \dots) \rightarrow r_t.$$

This flexibility is important for the planned LLM component. An LLM-generated reward does not need to modify the environment implementation; it only needs to produce a reward function that conforms to the defined interface.

The intended architecture is therefore

$$\begin{array}{c} \text{Task Description} \rightarrow \text{LLM} \rightarrow \text{Reward Representation} \\ \rightarrow R_{\text{custom}} \rightarrow \text{RewardCustom} \rightarrow \text{ChessboardGymEnv} \end{array}$$

while the PPO implementation remains independent of the specific reward definition.

1.8 Reward Function as a Pluggable Component

Several reward configurations were developed during the grid-world experiments. The distance-progress reward, for example, can be expressed as

$$R(s_t, a_t, s_{t+1}) = \begin{cases} +1, & s_{t+1} \text{ is a goal,} \\ -1, & s_{t+1} \text{ is a trap,} \\ \beta [d(s_t, G) - d(s_{t+1}, G)] - c, & \text{otherwise,} \end{cases}$$

where $d(s, G)$ denotes the Manhattan distance to the closest goal, β controls the contribution of progress toward the goal, and c represents the step penalty.

For the current implementation,

$$\beta = 0.1, \quad c = 0.01.$$

The architectural significance is that the reward can be replaced without modifying the PPO implementation. For example,

```
RewardCustom(env, reward_v4)
```

can later be replaced by

```
RewardCustom(env, reward_LLM).
```

The PPO agent therefore does not need to know whether the reward was manually defined, selected from a predefined reward library, or generated by an LLM.

1.9 Environment Generation

The Gymnasium adaptation was integrated with the existing environment generation mechanism.

A configuration specifies properties such as:

- base environment;
- number of goals;
- number of traps;
- number of generated environments;
- generation type.

The generator creates independent copies of the underlying `Chessboard`, randomises them, and converts each instance into a `ChessboardGymEnv`.

Two generation modes are currently used:

1. random environments;
2. blocked-path environments designed to require detours.

The generated environments are subsequently shuffled before training.

The resulting pipeline is

configuration $\rightarrow$ Chessboard copy $\rightarrow$ randomisation
 $\rightarrow$ Gymnasium adaptation $\rightarrow$ reward wrapper $\rightarrow$ training set.

This allows PPO to train on a distribution of environment instances rather than on a single fixed board, reducing the possibility of simply memorising one environment configuration.

1.10 Vectorised Training

After conversion to Gymnasium environments, the environments can be executed through Gymnasium’s vectorised environment interface. The current implementation uses `SyncVectorEnv`.

Instead of processing one environment at a time,

$$s_t^{(1)} \rightarrow a_t^{(1)} \rightarrow s_{t+1}^{(1)},$$

the PPO training loop can process a batch of environments:

$$\begin{bmatrix} s_t^{(1)} \\ s_t^{(2)} \\ \vdots \\ s_t^{(N)} \end{bmatrix} \rightarrow \begin{bmatrix} a_t^{(1)} \\ a_t^{(2)} \\ \vdots \\ a_t^{(N)} \end{bmatrix} \rightarrow \begin{bmatrix} s_{t+1}^{(1)} \\ s_{t+1}^{(2)} \\ \vdots \\ s_{t+1}^{(N)} \end{bmatrix}.$$

This provides a standard mechanism for collecting experience from multiple independent environment instances during PPO training.

1.11 Integration With the PPO Agent

The PPO implementation was kept independent of the internal `Chessboard` representation. During normal interaction, the training loop requires only the Gymnasium interface:

1. obtain the initial observation from `reset()`;
2. select an action using the PPO policy;
3. pass the action to `step()`;
4. receive the next observation, reward, termination information, and metadata;
5. store the transition;
6. perform the PPO update after collecting the rollout.

Consequently, the PPO implementation does not need direct access to `Chessboard`, `CellBox`, or the internal board representation during environment interaction.

The custom grid-world therefore becomes one Gymnasium-compatible environment among potentially many other environments that can be used by the same PPO implementation.

1.12 Current Experimental Validation

The Gymnasium pipeline was validated using the generated training environments.

The current experiment generated

396

Gymnasium environments from seven environment configurations. These configurations included different board sizes, numbers of goals and traps, and explicitly blocked-path environments.

The environments were wrapped using the custom reward mechanism and trained with the implemented PPO agent.

The training configuration was:

- 1,000,000 environment transitions;
- rollout length of 256;
- $\gamma = 0.90$;
- $\lambda = 0.95$;
- clipping coefficient $\epsilon = 0.2$;
- learning rate 3×10^{-4} ;
- six PPO optimisation epochs per rollout.

The training process completed approximately 2,500 PPO updates.

For subsequent evaluation, 1,000 blocked-path environments were generated from the large-board configuration. Each environment was evaluated for ten episodes. The resulting overall success rate was

92.2%.

The evaluation showed that the learned policy could repeatedly reproduce successful trajectories during deterministic evaluation. The remaining failures were primarily associated with repetitive trajectories and episode timeouts.

This result should be interpreted as a validation of the adapted Gymnasium–vectorisation–PPO pipeline rather than as evidence that the final robot navigation problem has been solved. The evaluation distribution remains a controlled grid-world distribution and therefore does not capture the complexity of the final ROSbot navigation task.

1.13 Architectural Significance for the LLM Reward Component

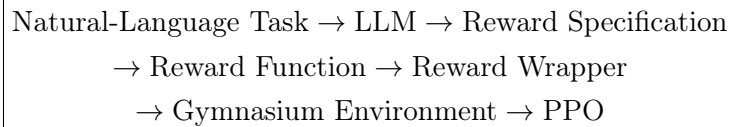
The Gymnasium implementation establishes an interface between four main components:

Environment Dynamics ↔ Gymnasium Interface ↔ Reward Interface ↔ RL Algorithm

The environment dynamics determine the result of an action. The Gymnasium adapter standardises how this interaction is exposed to the RL system. The reward interface determines how the resulting transition is evaluated, while PPO updates the policy using the collected experience.

This separation provides the integration point for the planned LLM component. The LLM can be introduced at the reward-definition level without requiring modifications to the environment dynamics or PPO implementation.

The intended final pipeline is therefore



The LLM consequently acts as a task-objective generator rather than as the controller itself. PPO remains responsible for learning the control policy through interaction with the environment.

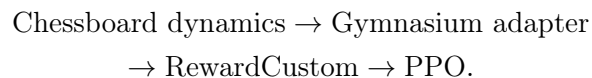
1.14 Current Limitations and Implementation Considerations

Several implementation considerations remain before the Gymnasium layer can be considered final.

First, the current implementation demonstrates Gymnasium compatibility but does not yet use `gym.register()` to register the custom environment under a persistent Gymnasium environment ID. Registration is therefore considered a future packaging step rather than part of the current implementation.

Second, the current notebook applies the reward function both inside the underlying `Chessboard` and through `RewardCustom` in some configurations. For example, the base environment may be instantiated with `reward_v4` and subsequently wrapped using `RewardCustom(reward_fn=reward_v4)`. When both functions are identical, this is redundant.

For the final LLM-based architecture, the cleaner design is to use the Gymnasium wrapper as the explicit reward injection point:



This keeps the generated reward separate from the environment dynamics.

Third, the reward wrapper should be treated as the definitive reward injection mechanism for the final architecture. This avoids coupling an LLM-generated reward to the implementation of the underlying environment.

Finally, the `RewardCustom` implementation in the current notebook should be cleaned before being presented as final code. In the notebook listing, the `step()` method appears outside the class because of an indentation issue. The intended design is clear, but the final implementation should contain the method inside the `RewardCustom` class.

1.15 Conclusion

The Gymnasium implementation extends the problem-specific grid-world into a standardised RL interface without replacing the existing environment architecture.

The `ChessboardGymEnv` translates the existing environment state, actions, and transitions into the Gymnasium API, while the `RewardCustom` wrapper provides a separate mechanism for replacing the reward function. This allows the resulting environments to be generated in large numbers, vectorised, and trained using the existing PPO implementation.

The current experiments generated 396 Gymnasium environments and achieved a 92.2% success rate on a subsequent evaluation of 1,000 blocked-path environments with ten episodes per environment. These results validate the operation of the adapted Gymnasium–vectorisation–PPO pipeline within the controlled grid-world setting.

The main significance of this implementation for the thesis is the separation between environment dynamics, reward definition, and policy learning. This provides a defined integration point for the planned LLM component, which can later generate reward specifications without modifying the environment dynamics or PPO algorithm.

The next stage is therefore to use this reward interface to integrate and evaluate LLM-generated reward functions against the established hand-crafted reward functions, before transferring the approach to the ROSbot navigation environment.